

Creating interactive online visualizations with Trame

2025-Nov-18

Alex Razoumov <alex.razoumov@westdri.ca>

Introduction

What is trame?

- Pronounced [trΛm]
- trame is an open-source platform for creating scientific web applications
- Created by Kitware: v1 in September 2021, now on v3.10
- Provides a unified client/server model with a Python interface
- Built on top of VTK and ParaView, provides widgets supporting many other plotting packages

Disclaimer

- Started looking into frame ~3 weeks ago, to prepare for this webinar
 - I have no connection to Kitware
 - before this deep dive I used to think of frame as a way to write VTK.js code (in-browser visualization with VTK) from Python – but it's a lot more than that!
- Think of frame as a server-connected web dashboard into which you can plug different components:
 - some mix of VTK and vtk.js (via an adaptor) scripts
 - ParaView scripts
 - Matplotlib scripts
 - Plotly scripts
 - etc
- All Python scripts run on the server backend
- Interactive graphical widgets rendered either on the server (and displayed in your browser) or directly in your browser

Evolution of VTK

- VTK (Visualization Toolkit)
 - C++ library, initial release in 1993
 - handles I/O, data processing, rendering
 - interpreted interface layer in Python (started in late 1990s, full API coverage by ~2018)
 - runs server-side only (C++ or Python)
- Vtk.js is a rewrite of (most of) VTK in JavaScript
 - runs client-side only (in the browser), started in 2016
 - rendering limited by the browser
 - somewhat limited processing and I/O
- Previous (pre-2021) state-of-the-art web visualization
 - standalone vtk.js applications, e.g. ParaView Glance or ParaView Divvy
 - vtk.js applications interacting with a remote ParaView server using ParaViewWeb framework, e.g.
 - ParaView Visualizer (recreating full ParaView functionality on the web)
 - ParaView Lite (lighter workflows than ParaView Visualizer)
- In all these cases you create web apps in JavaScript, or use existing apps

Trame architecture

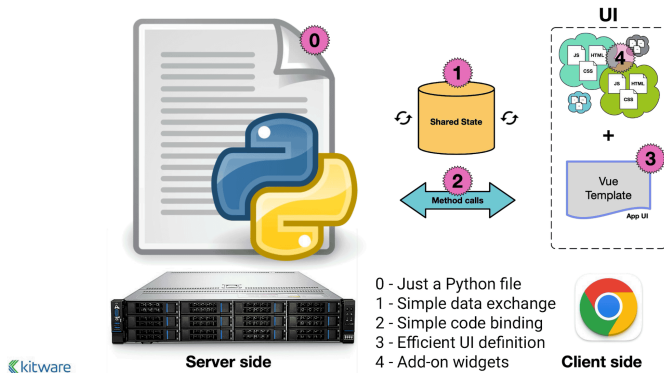


Figure 1: By Kitware, Inc.

- Single backend Python code on the server (cloud, HPC, Jupyter, desktop app)
- UI (layout/buttons/...), via Vuetify toolkit on top of Vue.js, nicely abstracted
- **State variables** automatically synced between the backend and the frontend (UI)
- **Controller actions** called from (1) Python, (2) frontend, or (3) state changes

Links

- Front page <https://kitware.github.io/trame>
 - click on any example to get its source code
- Source code <https://github.com/Kitware/trame>
 - check examples/
- Tutorial <https://kitware.github.io/trame/guide/tutorial>
 - excellent self-guided tour, probably the best starting point
 - today I will show many examples from their tutorial
- Documentation <https://trame.readthedocs.io>
 - use the left-side menu: trame API, client API, ...
- Discussion forum <https://github.com/Kitware/trame/discussions>

Workflows

Local installation (assuming uv)

```
uv venv ~/env-trame --python 3.12 # create a new virtual env
source ~/env-trame/bin/activate
uv pip install trame               # trame core
uv pip install trame-vuetify       # UI widgets
uv pip install trame-components    # other components to work with VTK/ParaView scenes
uv pip install trame-vtk           # VTK widgets for trame
uv pip install vtk                 # if running plain VTK code
uv pip install trame-matplotlib    # matplotlib widgets for trame
uv pip install trame-plotly        # plotly widgets for trame
...
deactivate
```

- If you want to run trame inside Jupyter notebooks, also add:

```
uv pip install setuptools jupyterlab
```

Layouts

- Study the code 00_setup/app.py

```
source ~/env-trame/bin/activate
cd ~/tmp/trame-tutorial
python 00_setup/app.py           # use default port 8080
python 00_setup/app.py --port 1234 # use specific port
python 00_setup/app.py --server  # prevent your browser from opening automatically
```

- Change SinglePageLayout to SinglePageWithDrawerLayout

Add a simple Matplotlib widget

- Matplotlib is a popular plotting library for Python
 - produces static plots
- Study and run the code `06_matplotlib/line1-static.py`
- Can hide `v3.VContainer` line
- Check [v3.VContainer documentation](#)

Add a state variable

- `state = server.state` is where you store data that can change dynamically and is automatically synced between the backend (Python on the server) and the frontend (browser UI)
- You can watch for changes in state on the backend using the `@state.change` decorator that will run a callback function
 - callback functions are defined with `@state.change`
- Study and run the code `06_matplotlib/line2-slider.py`

```
v3.VSlider(v_model=("wavenumber", defaultWave), min=0.1, max=5, step=0.1,  
          hide_details=True, density="compact", style="max-width: 300px;")  
v3.VBtn("Reset", click=lambda: setattr(state, "wavenumber", defaultWave))
```

```
@state.change("wavenumber")  
def update_plot(wavenumber, **kwargs):    # callback function  
    line.set_ydata(np.sin(wavenumber*x))  
    fig.canvas.draw_idle()                # redraw locally in matplotlib  
    plot.update()                          # notify Trame to refresh image in browser
```

Interactive slice through a 3D function

3D sine envelope function

$$f(x_1, x_2, x_3) = \sum_{i=1}^2 \left(\frac{\sin^2 \left(\sqrt{\xi_{i+1}^2 + \xi_i^2} \right) - 0.5}{[0.001(\xi_{i+1}^2 + \xi_i^2) + 1]^2} + 0.5 \right)$$
$$\xi_i \equiv 15(x_i - 0.5)$$

- Code in 06_matplotlib/slice2-streamlined.py
- Define slice state variable
- Redraw the image with:

```
@state.change("slice")          # watch for slice changes
def update_plot(slice, **kwargs):
    ax.imshow(data[:, :, slice], origin="lower")
    fig.canvas.draw_idle()      # redraw locally in matplotlib
    plot.update()               # notify Trame to refresh image in browser
```

Plotly widgets

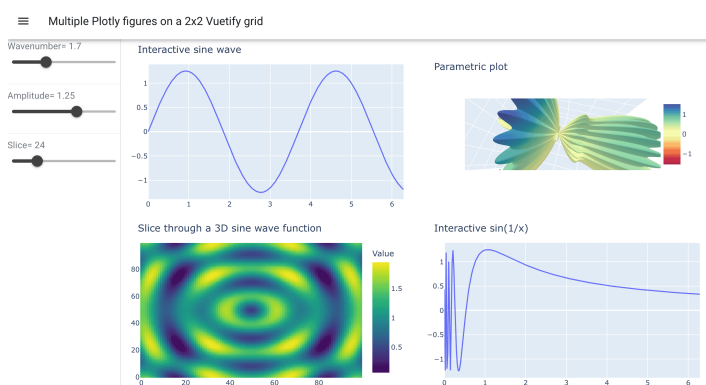
- Plotly is another popular plotting library for Python
 - produces interactive HTML5 plots
- Study and run the code 07_plotly/1-line-slider.py
 - single wavenumber slider
- Add a second slider in 07_plotly/2-line-twoSliders.py
 - one slider per variable
 - both variables are input to the callback function to update the plot:

```
@state.change("wavenumber", "amplitude")
def update_plot(wavenumber, amplitude, **kwargs):
    fig.data[0].y = amplitude * np.sin(wavenumber * x)
    plot.update() # refresh Plotly widget in the browser
```

- Move both sliders to the side drawer 07_plotly/3-line-addDrawer.py

Example of a more complex 2x2 layout

- Code in 07_plotly/4-dashboard2x2.py
 - 4 separate Plotly figures, including an interactive 3D plot
 - 2 callback functions
 - 2 familiar sliders (controlling fig1 and fig2)
 - one additional slider for a 3D slice position in fig3



VTK + trame

VTK can be integrated with various levels of granularity:

1. local vtk.js (client-side only)
2. hybrid: remote VTK for processing (creating meshes) and local vtk.js for rendering
3. remote VTK for processing and rendering on the server

Client-side rendering with vtk.js only

- Code in 01_no_vtk/client.py
- No traditional Python VTK code in this example
- Import a lightweight adaptor to present VTK objects to the UI:

```
from trame.widgets import vtk
```

- `vtk.VtkAlgorithm` handles the state variable as an argument (no explicit callback function)
- Run the code – all rendering is done on the client
- Add a second VTK object:

```
with vtk.VtkGeometryRepresentation():  
    vtk.VtkAlgorithm(vtk_class="vtkSphereSource")
```

VTk rendering

- Python interface to VTK provides the full functionality of the C++ API
 - handles I/O, data processing, rendering
 - Usual steps to render an object in VTK
 - should be familiar to anyone with some VTK experience in C++ or Python
1. Create a renderer, then a render window, and link them together by adding the renderer to the render window (a render window can hold multiple renderers)
 2. Create an interactor (which provides a platform-independent interaction mechanism for mouse/key events), connect it to the render window, and ensure it tracks the camera
 3. Create a source, e.g. a cone
 4. Create a mapper (a graphical representation) and set the cone source as its input
 5. Create an actor and assign the mapper to it
 6. Add the actor to the renderer (a renderer can hold multiple actors), then reset the camera to frame the scene
- Check the code in `01_vtk/coneCylinder.py`
 - two objects, each with its own mapper and actor
 - do either local (vtk.js on the client from received geometry) or remote (on the server and send images to the client) rendering

Render a VTK dataset from a file

- Code in `01_vtk/sine1-static.py`
 - loads `sineEnvelope.vti` file on the server
 - applies an Outline Filter, creates a outline mapper, then an outline actor
 - applies a Contour Filter at $\rho = 0.15$, creates a contour mapper, then a contour actor
 - adds both actors to the renderer
- Local rendering is noticeably faster even for this small dataset
 - local rendering: working with geometry on the client
 - remote rendering: receiving pixelated images
- Add a contour slider and a callback function – see `01_vtk/sine2-slider.py`

```
@state.change("contourValue")
def update_contour(contourValue, **kwargs):
    contour.SetValue(0, float(contourValue))
    view.update()
```

Controller actions

- Functions that can be called from Python or from the frontend UI (e.g. a button click) or from state changes
 - defined with `@ctrl.add` decorator
- Example in `03_html/click.py`

```
from trame.app import get_server
from trame.ui.vuetify3 import SinglePageLayout
from trame.widgets import vuetify3 as v3

server = get_server(client_type = "vue3")
ctrl = server.controller

@ctrl.add("printout")
def reset_plot(): # this function gets triggered when you click Print
    print("Another line!")

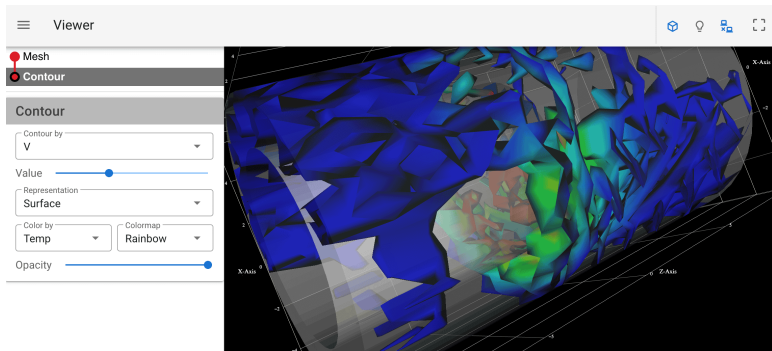
# ctrl.printout() # syntax to call it in the main Python code

with SinglePageLayout(server) as layout: # create a UI
    with layout.content:
        v3.VBtn("Print", click=ctrl.printout)

server.start()
```

Complex VTK + trame visualization app

- Code 04_application/coneMeshContour.py adapted from the tutorial
 - reads an Unstructured Grid file on the server
 - creates an entire visualization pipeline in the drawer
 - packs pipeline elements into collapsible Vuetify UI cards
 - good usage example for `vuetify.VCol()` and `vuetify.VRow()`
 - the original Python code is 500+ lines



ParaView rendering

- Code 05_paraview/cone.py adapted from the tutorial
- To run it, use ParaView's Python, pointing to the correct venv with `--venv` instead of `source`
 - `paraview.VtkRemoteView` uses ParaView's Python VTK pipeline underneath to render the image on the server, delivers this image to the browser, works right out of the box
 - `paraview.VtkLocalView` **should** deliver the cone's geometry to the browser for rendering via `vtk.js`, but apparently it does not, and the view appears empty ... not sure why

```
cd ~/tmp/trame-tutorial
pvpython 05_paraview/cone.py --venv ~/env-trame
```

- ParaView Python workflows are much more compact than VTK Python workflows
- To learn more about ParaView Python scripting, use the Trace Tool in ParaView's GUI

ParaView state loader via the command line

- Code 05_paraview/stateLoader.py adapted from the tutorial
- Open local ParaView GUI
 - load ~/training/paraviewWorkshop/data/sineEnvelope.nc
 - create a interactive visualization with several layers
 - save the state as 05_paraview/demo.pvsm
- Run the trame script:

```
pvpython 05_paraview/stateLoaderFromFlag.py --venv ~/env-trame --data 05_paraview/demo.pvsm
```

- Temporarily remove the data file sineEnvelope.nc from its location to verify that demo.pvsm uses live data

ParaView state loader via upload

- Code 05_paraview/uploadState.py
- Run this script:

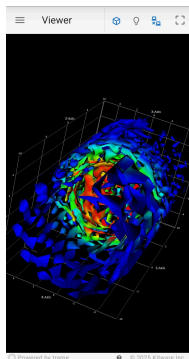
```
pvpython 05_paraview/uploadState.py --venv ~/env-trame
```

- It'll prompt you to upload the state (.pvsm) file – use 05_paraview/demo.pvsm
- We save the content of this uploaded file bytes = file["content"] into a remote file uploaded_state.pvsm on the server
- It'll read the data file sineEnvelope.nc on the server side
- For a flashier demo, expand this script to upload a data file as well
 - the entire visualization could be run without any pre-existing files on the server (upload both .pvsm and .nc)
 - good starting point in documentation <https://kitware.github.io/trame/examples/core/files.html>

Server on a VM, client on a phone

```
# running this in a VM on Cedar Cloud
source ~/env-trame/bin/activate
cd ~/tmp/trame-tutorial
python 04_application/coneMeshContour.py --host 0.0.0.0 --port 8080 --server
```

- Point my phone's browser to VM's port 8080 to view the visualization



Multi-user setup

- By default, for development purposes, a simple `trame.app` serves a single user
- To scale to multiple users in a production environment, you need a setup that isolates user sessions while allowing a single server process to handle all connections
 - apparently, several ways to do this using third-party web development tools
 - not being a web developer, I have not spent time on this

Visualizer in frame

- All of ParaView's functionality in the browser
- Originally written as a ParaViewWeb application (JavaScript frontend talking to a remote ParaView server)
- Kitware has an implementation in frame
 - need to request access (work in progress? I have not tried it)
 - few other complete frame apps on the same page

Summary

Trame summary

- Enables any Python developer to create stunning interactive web visualizations using their favourite data-processing engine (numpy, pandas, ...)
- Provides integration with popular Python analysis/visualization libraries such as VTK, ParaView, Matplotlib, Plotly, Altair, Vega, etc.
- A trame app is a single Python application
 - can run in the cloud, on HPC, within Jupyter, or bundled locally like any desktop app
 - displays everywhere via the web (phone, tablet, laptop)
- Utilizes a dedicated stateful server process per client
- Uses Vuetify UI toolkit, a no-design-skills-required open-source UI library for JavaScript / TypeScript, but expresses the UI design in plain Python code
- Excellent documentation
 - tutorial <https://kitware.github.io/trame/guide/tutorial>
 - official documentation <https://trame.readthedocs.io>
 - discussion forum <https://github.com/Kitware/trame/discussions>
 - found Sebastien Jourdain's 2022-Oct [Trame Intro \(for developers\) course](#) very useful